

```

from google.colab import files
files.upload()

<IPython.core.display.HTML object>

Saving kaggle.json to kaggle.json

{'kaggle.json':
b'{"username":"saiparayan","key":"e18104a712895267cad53b1897a97d59"}'}

import os

os.makedirs("/root/.kaggle", exist_ok=True)

!cp kaggle.json /root/.kaggle/
!chmod 600 /root/.kaggle/kaggle.json

!kaggle datasets download -d andradaolteanu/gtzan-dataset-music-genre-
classification

Dataset URL: https://www.kaggle.com/datasets/andradaolteanu/gtzan-
dataset-music-genre-classification
License(s): other
Downloading gtzan-dataset-music-genre-classification.zip to /content
100% 1.21G/1.21G [01:11<00:00, 18.3MB/s]

!unzip -q gtzan-dataset-music-genre-classification.zip

import os

print(os.listdir())

['.config', 'kaggle.json', 'Data', 'gtzan-dataset-music-genre-
classification.zip', 'sample_data']

import os

print(os.listdir('/content/Data'))

['features_3_sec.csv', 'features_30_sec.csv', 'images_original',
'genres_original']

print(os.listdir('/content/Data/genres_original'))

['classical', 'reggae', 'disco', 'jazz', 'rock', 'metal', 'pop',
'blues', 'country', 'hiphop']

print(os.listdir('/content/Data/genres_original/blues')[:5])

['blues.00058.wav', 'blues.00074.wav', 'blues.00000.wav',
'blues.00006.wav', 'blues.00046.wav']

```

```

import os
import numpy as np
import librosa
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import confusion_matrix, classification_report

from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import (
    Conv2D,
    MaxPooling2D,
    BatchNormalization,
    Dropout,
    Reshape,
    LSTM,
    Dense
)

# =====
# PARAMETERS
# =====

DATASET_PATH = "/content/Data/genres_original"      # Folder containing
genre folders
SAMPLE_RATE = 22050
DURATION = 30                                     # seconds
N_MELS = 128
FIXED_TIME_FRAMES = 130

# =====
# FEATURE EXTRACTION
# =====

def extract_mel_spectrogram(file_path):
    """
    Convert audio file to fixed-size Mel Spectrogram.
    """

    signal, sr = librosa.load(
        file_path,
        sr=SAMPLE_RATE,
        duration=DURATION
    )

    mel_spec = librosa.feature.melspectrogram(
        y=signal,

```

```

        sr=sr,
        n_mels=N_MELS
    )

    mel_db = librosa.power_to_db(
        mel_spec,
        ref=np.max
    )

    # Ensure equal size for all samples
    if mel_db.shape[1] < FIXED_TIME_FRAMES:
        pad_width = FIXED_TIME_FRAMES - mel_db.shape[1]
        mel_db = np.pad(
            mel_db,
            ((0, 0), (0, pad_width)),
            mode='constant'
        )
    else:
        mel_db = mel_db[:, :FIXED_TIME_FRAMES]

    return mel_db

# =====
# LOAD DATASET
# =====

X = []
y = []

genres = sorted(os.listdir(DATASET_PATH))

for genre in genres:
    genre_path = os.path.join(DATASET_PATH, genre)

    if not os.path.isdir(genre_path):
        continue

    print(f"Processing {genre}")

    for audio_file in os.listdir(genre_path):
        if audio_file == "jazz.00054.wav":
            continue

        file_path = os.path.join(
            genre_path,
            audio_file
        )

        try:

```

```

        feature = extract_mel_spectrogram(file_path)

        X.append(feature)
        y.append(genre)

    except Exception as e:
        print("Skipped:", file_path)
        print(e)

X = np.array(X)
y = np.array(y)

print("Dataset Shape:", X.shape)
print("X shape:", X.shape)
print("y shape:", y.shape)

print("\nUnique labels:")
print(np.unique(y))

print("\nNumber of classes:")
print(len(np.unique(y)))

# =====
# ENCODE LABELS
# =====

encoder = LabelEncoder()
y_encoded = encoder.fit_transform(y)

num_classes = len(np.unique(y_encoded))

y_encoded = to_categorical(
    y_encoded,
    num_classes=num_classes
)

# Add channel dimension
X = X[..., np.newaxis]

# =====
# TRAIN TEST SPLIT
# =====

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y_encoded,
    test_size=0.2,
    random_state=42,
    stratify=y_encoded
)

```

```

# =====
# CNN-RNN MODEL
# =====

model = Sequential()

# CNN Block 1
model.add(
    Conv2D(
        32,
        (3, 3),
        activation='relu',
        padding='same',
        input_shape=(N_MELS, FIXED_TIME_FRAMES, 1)
    )
)

model.add(MaxPooling2D((2, 2)))
model.add(BatchNormalization())

# CNN Block 2
model.add(
    Conv2D(
        64,
        (3, 3),
        activation='relu',
        padding='same'
    )
)

model.add(MaxPooling2D((2, 2)))
model.add(BatchNormalization())

# CNN Block 3
model.add(
    Conv2D(
        128,
        (3, 3),
        activation='relu',
        padding='same'
    )
)

model.add(MaxPooling2D((2, 2)))
model.add(BatchNormalization())

# Convert CNN output for LSTM
model.add(
    Reshape(
        (

```

```

        16,      # time steps
        16 * 128 # features
    )
)
)

# RNN Block
model.add(
    LSTM(
        128,
        return_sequences=False
    )
)

model.add(Dropout(0.3))

# Fully Connected Layers
model.add(Dense(128, activation='relu'))
model.add(Dropout(0.3))

model.add(
    Dense(
        num_classes,
        activation='softmax'
    )
)

# =====
# COMPILE
# =====

model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

model.summary()

# =====
# TRAIN
# =====

history = model.fit(
    X_train,
    y_train,
    validation_split=0.2,
    epochs=30,
    batch_size=32
)

```

```

# =====
# EVALUATION
# =====

test_loss, test_acc = model.evaluate(
    X_test,
    y_test
)

print("\nTest Accuracy:", test_acc)

# =====
# PREDICTIONS
# =====

predictions = model.predict(X_test)

y_pred = np.argmax(predictions, axis=1)
y_true = np.argmax(y_test, axis=1)

# =====
# CLASSIFICATION REPORT
# =====

print(
    classification_report(
        y_true,
        y_pred,
        target_names=encoder.classes_
    )
)

# =====
# CONFUSION MATRIX
# =====

cm = confusion_matrix(
    y_true,
    y_pred
)

plt.figure(figsize=(10, 8))

sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=encoder.classes_,

```

```

        yticklabels=encoder.classes_
    )

plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()

# =====
# ACCURACY CURVE
# =====

plt.figure(figsize=(8,5))

plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])

plt.title("Model Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")

plt.legend([
    "Training",
    "Validation"
])

plt.show()

# =====
# LOSS CURVE
# =====

plt.figure(figsize=(8,5))

plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])

plt.title("Model Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")

plt.legend([
    "Training",
    "Validation"
])

plt.show()

Processing blues
Processing classical
Processing country

```



```
Processing disco
Processing hiphop
Processing jazz
Processing metal
Processing pop
Processing reggae
Processing rock
Dataset Shape: (999, 128, 130)
X shape: (999, 128, 130)
y shape: (999,)
```

```
Unique labels:
['blues' 'classical' 'country' 'disco' 'hiphop' 'jazz' 'metal' 'pop'
 'reggae' 'rock']
```

```
Number of classes:
10
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/
convolutional/base_conv.py:113: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential
models, prefer using an `Input(shape)` object as the first layer in
the model instead.
```

```
super().__init__(activity_regularizer=activity_regularizer,
**kwargs)
```

```
Model: "sequential_2"
```

Layer (type) Param #	Output Shape	
conv2d_6 (Conv2D) 320	(None, 128, 130, 32)	
max_pooling2d_6 (MaxPooling2D) 0	(None, 64, 65, 32)	
batch_normalization_6 128 (BatchNormalization)	(None, 64, 65, 32)	
conv2d_7 (Conv2D) 18,496	(None, 64, 65, 64)	

0	max_pooling2d_7 (MaxPooling2D)	(None, 32, 32, 64)
256	batch_normalization_7 (BatchNormalization)	(None, 32, 32, 64)
73,856	conv2d_8 (Conv2D)	(None, 32, 32, 128)
0	max_pooling2d_8 (MaxPooling2D)	(None, 16, 16, 128)
512	batch_normalization_8 (BatchNormalization)	(None, 16, 16, 128)
0	reshape_2 (Reshape)	(None, 16, 2048)
1,114,624	lstm_2 (LSTM)	(None, 128)
0	dropout_4 (Dropout)	(None, 128)
16,512	dense_4 (Dense)	(None, 128)
0	dropout_5 (Dropout)	(None, 128)
1,290	dense_5 (Dense)	(None, 10)

Total params: 1,225,994 (4.68 MB)

Trainable params: 1,225,546 (4.68 MB)

Non-trainable params: 448 (1.75 KB)

Epoch 1/30

20/20 ————— 6s 70ms/step - accuracy: 0.1894 - loss: 2.2230 - val_accuracy: 0.1437 - val_loss: 2.3577

Epoch 2/30

20/20 ————— 1s 34ms/step - accuracy: 0.2754 - loss: 1.9809 - val_accuracy: 0.1063 - val_loss: 2.4371

Epoch 3/30

20/20 ————— 1s 37ms/step - accuracy: 0.3349 - loss: 1.8177 - val_accuracy: 0.0375 - val_loss: 2.4290

Epoch 4/30

20/20 ————— 1s 41ms/step - accuracy: 0.3725 - loss: 1.7092 - val_accuracy: 0.1125 - val_loss: 2.6225

Epoch 5/30

20/20 ————— 1s 35ms/step - accuracy: 0.3975 - loss: 1.6304 - val_accuracy: 0.1437 - val_loss: 2.5739

Epoch 6/30

20/20 ————— 1s 32ms/step - accuracy: 0.3818 - loss: 1.5971 - val_accuracy: 0.2688 - val_loss: 2.1773

Epoch 7/30

20/20 ————— 1s 32ms/step - accuracy: 0.4413 - loss: 1.5246 - val_accuracy: 0.1875 - val_loss: 2.3466

Epoch 8/30

20/20 ————— 1s 32ms/step - accuracy: 0.4695 - loss: 1.4584 - val_accuracy: 0.1625 - val_loss: 3.1186

Epoch 9/30

20/20 ————— 1s 32ms/step - accuracy: 0.5023 - loss: 1.3879 - val_accuracy: 0.3750 - val_loss: 1.7637

Epoch 10/30

20/20 ————— 1s 31ms/step - accuracy: 0.5540 - loss: 1.2506 - val_accuracy: 0.2688 - val_loss: 2.8980

Epoch 11/30

20/20 ————— 1s 31ms/step - accuracy: 0.5649 - loss: 1.2236 - val_accuracy: 0.3625 - val_loss: 1.6701

Epoch 12/30

20/20 ————— 1s 32ms/step - accuracy: 0.6056 - loss: 1.1317 - val_accuracy: 0.4563 - val_loss: 1.4953

Epoch 13/30

20/20 ————— 1s 31ms/step - accuracy: 0.6275 - loss: 1.0450 - val_accuracy: 0.3500 - val_loss: 1.8663

Epoch 14/30

20/20 ————— 1s 32ms/step - accuracy: 0.6322 - loss:

```
1.0249 - val_accuracy: 0.3625 - val_loss: 1.8281
Epoch 15/30
20/20 _____ 1s 40ms/step - accuracy: 0.6667 - loss:
0.9486 - val_accuracy: 0.4750 - val_loss: 1.4452
Epoch 16/30
20/20 _____ 1s 40ms/step - accuracy: 0.7167 - loss:
0.8605 - val_accuracy: 0.2875 - val_loss: 2.5509
Epoch 17/30
20/20 _____ 1s 40ms/step - accuracy: 0.7387 - loss:
0.7609 - val_accuracy: 0.2937 - val_loss: 2.3780
Epoch 18/30
20/20 _____ 1s 32ms/step - accuracy: 0.7746 - loss:
0.6888 - val_accuracy: 0.4563 - val_loss: 1.7529
Epoch 19/30
20/20 _____ 1s 36ms/step - accuracy: 0.7966 - loss:
0.6000 - val_accuracy: 0.3625 - val_loss: 2.2042
Epoch 20/30
20/20 _____ 1s 41ms/step - accuracy: 0.8404 - loss:
0.5295 - val_accuracy: 0.4812 - val_loss: 1.6737
Epoch 21/30
20/20 _____ 1s 38ms/step - accuracy: 0.8404 - loss:
0.5226 - val_accuracy: 0.4187 - val_loss: 1.9029
Epoch 22/30
20/20 _____ 1s 35ms/step - accuracy: 0.8419 - loss:
0.4766 - val_accuracy: 0.4000 - val_loss: 1.8712
Epoch 23/30
20/20 _____ 1s 32ms/step - accuracy: 0.8732 - loss:
0.3990 - val_accuracy: 0.4688 - val_loss: 1.7818
Epoch 24/30
20/20 _____ 1s 32ms/step - accuracy: 0.8811 - loss:
0.3817 - val_accuracy: 0.4375 - val_loss: 1.7865
Epoch 25/30
20/20 _____ 1s 32ms/step - accuracy: 0.8779 - loss:
0.3506 - val_accuracy: 0.3187 - val_loss: 2.8582
Epoch 26/30
20/20 _____ 1s 32ms/step - accuracy: 0.9186 - loss:
0.3105 - val_accuracy: 0.4187 - val_loss: 1.7357
Epoch 27/30
20/20 _____ 1s 32ms/step - accuracy: 0.9202 - loss:
0.2931 - val_accuracy: 0.5188 - val_loss: 1.5749
Epoch 28/30
20/20 _____ 1s 32ms/step - accuracy: 0.9171 - loss:
0.2559 - val_accuracy: 0.4000 - val_loss: 2.7685
Epoch 29/30
20/20 _____ 1s 32ms/step - accuracy: 0.9296 - loss:
0.2459 - val_accuracy: 0.5312 - val_loss: 1.5499
Epoch 30/30
20/20 _____ 1s 32ms/step - accuracy: 0.9311 - loss:
0.2306 - val_accuracy: 0.4875 - val_loss: 1.8215
```

7/7 _____ 0s 22ms/step - accuracy: 0.4650 - loss:
1.7409

Test Accuracy: 0.4650000035762787

7/7 _____ 0s 37ms/step

	precision	recall	f1-score	support
blues	0.45	0.50	0.48	20
classical	0.80	0.60	0.69	20
country	0.35	0.35	0.35	20
disco	0.30	0.35	0.33	20
hiphop	0.60	0.30	0.40	20
jazz	0.47	0.75	0.58	20
metal	1.00	0.35	0.52	20
pop	0.59	0.80	0.68	20
reggae	0.41	0.55	0.47	20
rock	0.12	0.10	0.11	20
accuracy			0.47	200
macro avg	0.51	0.46	0.46	200
weighted avg	0.51	0.47	0.46	200





